

Day 46: Pod Priority, PriorityClass, and Preemption in Kubernetes | CKA Course 2025

Video reference for Day 46 is the following:



Table of Contents

- Introduction
- Why Pod Priority and Preemption Exist
- What Is a PriorityClass and How to Use It
- Understanding Preemption and Scheduler Behavior
- Special Notes and Advanced Insights
- Demo: Pod Priority and Preemption in Action
 - Step 1: Inspect Your Cluster's Capacity and Existing PriorityClasses
 - Step 2: Create PriorityClasses
 - * Optional Field: `preemptionPolicy: Never`
 - Step 3: Deploy Low-Priority Pods
 - Step 4: Deploy High-Priority Pods and Observe Preemption
 - Key Takeways

- Conclusion
 - References
-

Introduction

In Kubernetes, not all workloads are created equal. Some applications are mission-critical, while others can be deprioritized during resource pressure. This lecture introduces the concept of **Pod Priority and Preemption**, a powerful mechanism in Kubernetes that enables intelligent scheduling and eviction based on workload importance.

By the end of this lecture, you'll understand how to define priority classes, how the Kubernetes scheduler respects these during both normal and constrained conditions, and how to test and observe preemption scenarios using simple yet realistic demos on a KIND cluster.

Whether you're deploying payment systems, batch jobs, or development dashboards, this knowledge will help you control workload fairness and resilience under pressure.

Why Pod Priority and Preemption Exist

In Kubernetes, not all workloads are equal — some are business-critical (e.g., payment processors), while others are non-critical (e.g., dev dashboards, batch jobs).

During a resource crunch, Kubernetes should **favor critical workloads** over less important ones. That's where **Pod Priority** comes in — it guides the **scheduler** on **which pods to prefer** when scheduling decisions are made.

But what happens when the cluster is already full and a critical workload needs to be scheduled?

That's when **Preemption** kicks in — Kubernetes **evicts lower-priority pods** to make room for **higher-priority ones**. This makes sure that your critical apps get the resources they need, even at the cost of non-essential workloads.

What Is a PriorityClass and How to Use It

To avoid hardcoding numeric priorities in each pod, Kubernetes provides a resource called **PriorityClass**.

Instead of specifying raw priority numbers per pod, you define multiple reusable classes like:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000
globalDefault: false
description: "Critical production workloads"
```

Then, in your workload manifest:

```
spec:
  priorityClassName: high-priority
```

Higher numeric value = higher priority. A pod with priority 1000 will be scheduled before one with 100. The valid value range for custom workloads is from -2,147,483,648 to 1,000,000,000.

Pods that do not define a priority class default to priority 0, unless you have a class marked as `globalDefault: true`.

Understanding Preemption and Scheduler Behavior

Preemption is Kubernetes' way of **reclaiming resources** for critical pods.

If a high-priority pod can't be scheduled due to insufficient resources:

- Kubernetes evaluates which lower-priority pods are running.
- It **evicts one or more low-priority pods** to free up space.
- The critical pod is then scheduled.

This eviction is **graceful**, and the evicted pods might be rescheduled later if resources become available.

Important: Preemption is only triggered when the scheduler has **no other way** to schedule a high-priority pod.

You can **prevent a pod from preempting others**, even if it has a high priority, by setting:

```
preemptionPolicy: Never
```

This is useful for workloads that should **not disrupt others**, even when marked important.

If both high-priority and low-priority pods are already running normally, and the cluster suddenly faces a resource crunch (e.g., due to node failure or CPU spikes), Kubernetes does **not proactively**

evict low-priority pods. Preemption is a **scheduler-driven mechanism** that only activates when a new pod—typically a high-priority one—fails to get scheduled due to insufficient resources. In that case, the scheduler looks for lower-priority pods to evict and make room. However, if no new pod is being scheduled, then even during resource pressure, **running pods are left untouched**. Runtime issues like CPU throttling or OOM kills are handled by the kubelet and not tied to priority or preemption.

Kubernetes assigns each pod a QoS class—**Guaranteed**, **Burstable**, or **BestEffort**—based on its CPU and memory **requests** and **limits**. This is **separate from Pod Priority**, and comes into play during **node-level resource pressure**, where higher QoS classes (like **Guaranteed**) are less likely to be evicted.

Special Notes and Advanced Insights

- **PriorityClass** is a **cluster-scoped resource** — once created, it can be used across all namespaces.
- You can check existing priority classes using:

```
kubectl get pc
```

Output:

NAME	VALUE	GLOBAL-DEFAULT
system-cluster-critical	2000000000	false
system-node-critical	2000001000	false

These built-in classes are reserved for **system components** like CoreDNS, kube-proxy, and are **outside the user-defined value range**.

- You can define any number of priority tiers:
 - **low-priority**: 100
 - **medium-priority**: 500
 - **high-priority**: 1000
 - **batch-job**: 50
 - etc.
- Common best practices:
 - Use meaningful names like **batch-low**, **api-medium**, **critical-high**.
 - Apply **preemptionPolicy**: **Never** to logging agents or stateful apps.
 - Use **globalDefault** only once per cluster.
 - Test preemption scenarios before using in production.

Demo: Pod Priority and Preemption in Action

In this demo, we'll walk through how Kubernetes uses **priority classes** to make intelligent scheduling and eviction decisions when resources are constrained. This setup will use a **single-node KIND cluster**, making resource behavior and pod preemption easier to observe.

If you have KIND installed, spin up a cluster using:

```
kind create cluster --name cka-2025
```

Step 1: Inspect Your Cluster's Capacity and Existing PriorityClasses

Start by checking your cluster's node details:

```
kubectl get nodes
```

You'll see output like:

NAME	STATUS	ROLES	AGE	VERSION
cka-2025-control-plane	Ready	control-plane	23m	v1.33.1

Now describe the node:

```
kubectl describe node cka-2025-control-plane
```

You'll notice this KIND node offers **11 vCPUs** in total (cpu: 11 under Allocatable).

Also inspect existing PriorityClass definitions:

```
kubectl get pc
```

Expected output:

NAME	VALUE	GLOBAL-DEFAULT	AGE
system-cluster-critical	2000000000	false	24m
system-node-critical	2000001000	false	24m

These are built-in Kubernetes priority classes used by control plane components.

Let's verify who is using these:

```
kubectl describe pod -n kube-system etcd-cka-2025-control-plane
```

```
kubectl describe pod -n kube-system kube-apiserver-cka-2025-control-plane
```

Both pods will show:

```
Priority: 2000001000
Priority Class Name: system-node-critical
```

Observation: Kubernetes reserves the **2 billion+ priority range** for system-critical components. Any custom PriorityClass you define must stay within the **maximum allowed value of 1 billion**.

If you attempt a higher value, you'll encounter:

error when creating "pc.yaml": PriorityClass in version "v1" cannot be handled... json: canna

Step 2: Create PriorityClasses

Let's define 3 different priority classes in a manifest 01-priorityclass.yaml:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000
globalDefault: false
description: "Critical production workloads that must preempt lower-priority pods if needed"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: medium-priority
value: 100
globalDefault: false
description: "Important batch or background workloads that can tolerate delay"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 10
globalDefault: true
description: "Default for development and non-critical test workloads"
```

Apply the manifest:

```
kubectl apply -f 01-priorityclass.yaml
```

The `low-priority` class is marked as `globalDefault: true`, so any workload that doesn't explicitly define a `priorityClassName` will automatically be assigned this class.

In Kubernetes, pod priorities are defined using `PriorityClass` objects, and the numeric value you assign determines the scheduling preference. For user-defined workloads, the valid range is from **-2,147,483,648** (minimum of a 32-bit signed integer) up to **1,000,000,000** (1 billion). Any attempt to assign a value outside this range will result in an error. However, Kubernetes itself reserves special priority classes for critical system components

like `kube-apiserver` and `etcd`. These use priority values **above 2,000,000,000**, such as 2000000000 for `system-cluster-critical` and 2000001000 for `system-node-critical`. These reserved values are not configurable and always take precedence over any user-defined priorities.

Kubernetes allows only **one** `PriorityClass` to be marked as `globalDefault: true` in a cluster. This ensures that any pod without an explicitly defined `priorityClassName` automatically inherits the priority value from this single default class. If you attempt to create more than one `PriorityClass` with `globalDefault: true`, the API server will reject it with a validation error.

Optional Field: `preemptionPolicy: Never`

By default, when a high-priority pod is scheduled and the cluster lacks sufficient resources, Kubernetes may **preempt** (evict) one or more lower-priority pods to make room. This behavior is controlled by the `preemptionPolicy` field in the `PriorityClass` resource.

You can override this behavior using:

`preemptionPolicy: Never`

This instructs the scheduler **not to evict** lower-priority pods, even if the high-priority pod cannot be scheduled. As a result, pods assigned to this `PriorityClass` will **fail to schedule** rather than cause disruption to existing workloads.

This is useful when:

- You want to **avoid terminating user-facing or batch jobs**.
- You prefer **manual intervention** for rescheduling.
- You're deploying **non-critical high-priority** pods that should not disrupt running applications.

Example:

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-non-disruptive
value: 900
globalDefault: false
preemptionPolicy: Never
description: "High priority, but should not preempt running pods"
```

Note: Setting `preemptionPolicy: Never` does not affect scheduling order when resources are available—it only prevents evictions during resource crunch scenarios.

By default, `preemptionPolicy` is set to `PreemptLowerPriority`, allowing higher-priority pods to evict lower-priority ones. To prevent this, set `preemptionPolicy: Never` so the pod will wait for resources instead of preempting others.

A `PriorityClass` influences scheduling order; higher-priority pods are always scheduled before lower-priority ones when resources are available. Even if `preemptionPolicy` is set to `Never`, the scheduler still prefers these pods without evicting others. This is useful when you want a workload prioritized during normal operation but without disrupting existing pods during resource crunch. In essence, priority affects scheduling; preemption controls eviction.

Step 3: Deploy Low-Priority Pods

Let's deploy 5 low-priority NGINX pods. Each pod will request **1 vCPU**.

```
# 02-low-priority-nginx.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lp-nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          resources:
            requests:
              memory: "64Mi"
              cpu: "1000m"
```

Apply it:

```
kubectl apply -f 02-low-priority-nginx.yaml
```

These pods will consume **5 vCPUs total**, plus approximately 0.950 vCPU already in use by the control plane and supporting pods.

Verify their assigned priority:

```
kubectl describe pod <lp-nginx-pod-name>
```

Output will confirm:

```
Priority:          10
Priority Class Name: low-priority
```

Also inspect the node again:

```
kubectl describe node cka-2025-control-plane
```

You'll observe CPU requests distributed among:

- Control plane components (about 950m)
- lp-nginx pods ($5 \times 1000\text{m} = 5000\text{m}$)

Total used: ~5.950 vCPUs

Step 4: Deploy High-Priority Pods and Observe Preemption

Now deploy another 5 replicas of NGINX, but this time with a **high priority class** and **3 vCPUs per pod** (total 15 vCPUs requested).

```
# 03-high-priority-nginx.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hp-nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      priorityClassName: high-priority
      containers:
        - name: nginx
          image: nginx:latest
          resources:
            requests:
```

```
memory: "64Mi"
cpu: "3000m"
```

Before applying, watch pods in real time:

```
kubectl get pods -w
```

Apply the deployment:

```
kubectl apply -f 03-high-priority-nginx.yaml
```

Since the node has only **11 vCPUs**, and 5.950 vCPUs are already used, the scheduler cannot fit new pods unless it **evicts existing low-priority pods**.

You will observe lp-nginx pods being terminated to free up resources, and hp-nginx pods being scheduled.

Verify:

```
kubectl describe node cka-2025-control-plane
```

Look for pods like:

```
default  hp-nginx-5746bb7b5c-xxxxx  3 (27%)  ...
default  lp-nginx-6c8f7d669b-xxxx  1 (9%)   ...
```

As more hp-nginx pods are scheduled, more lp-nginx pods are evicted.

Key Takeways:

- Kubernetes honors **pod priority** during both scheduling and eviction.
- When resources are insufficient, lower-priority pods are **preempted** to make room for higher-priority workloads.
- The PriorityClass abstraction lets you define clear scheduling rules across teams, apps, and environments.

Conclusion

Pod Priority and Preemption in Kubernetes enable fine-grained control over which workloads are prioritized when scheduling and which are evicted during resource scarcity. By leveraging **PriorityClass**, you avoid hardcoding priority values and instead define consistent policies for different workload types — critical, batch, dev, etc.

The use of **preemptionPolicy: Never** gives you additional control for non-disruptive workloads. This lecture has equipped you with practical insights and hands-on demos to confidently configure, test, and troubleshoot priority-based scheduling in real-world Kubernetes environments.

References

- [Kubernetes Documentation — Pod Priority and Preemption](#)
 - [API Reference — `PriorityClass` Spec](#)
 - [CKA Course GitHub Repo — CloudWithVarJosh/CKA-Certification-Course-2025](#)
 - [KIND — Kubernetes IN Docker](#)
-